

Patterns of Patterns

And why we need them

A peek inside

- The misery of having thousands of patterns.
- Local and distributed architectures are not dissimilar.
- Structure determines function.
- There are only so many elementary geometries.
- Which means that hundreds of patterns condense into several metapatterns.

Expectation vs Reality

What we were promised 30 years ago

Patterns as a ubiquitous language of software architecture.

Patterns as vessels for sharing and recording experience.

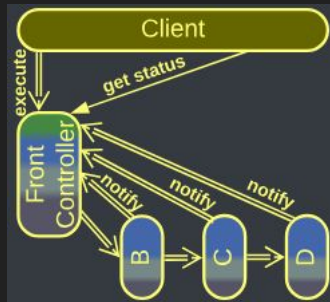
A pattern language as a construction set for software engineering.

There are too many patterns for any person to remember. Some patterns names are ambiguous.

Patterns are regularly reinvented or plagiarized, breaking the continuity of experience and debarring the sharing of knowledge between domains.

A few specialized pattern languages became widespread. No attempt at a generic pattern language has ever succeeded.

Which is the *Front Controller*



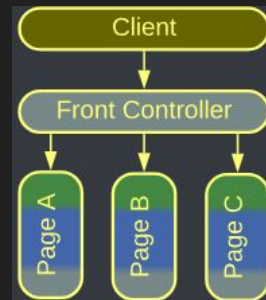
Neal Ford and
Mark Richards, 2021



?

A service that tracks the status of requests by receiving notifications from a *Pipeline*.

Martin Fowler et al.,
2002

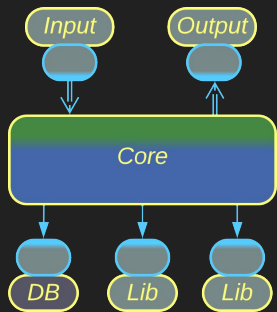


A part of a backend *MVC* which parses client requests and calls page classes.

Is Mark Richards wrong? He is the author of the most modern anthology of architectural styles!

There are over 2000 patterns! No one can remember them!

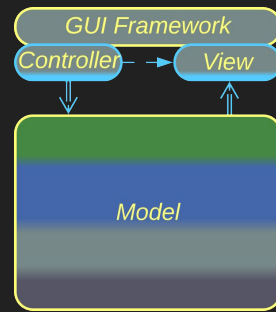
Patterns are reinvented...



Hexagonal Architecture
(Ports and Adapters),
Alistair Cockburn,
2005



Model-View-Controller
(MVC),
Trygve Reenskaug,
1979



Allows for running the same *core* logic in different setups.

The external components can be changed without affecting the *core*.

Protection from vendor lock-in.

Supports multiple views of a single *model*.

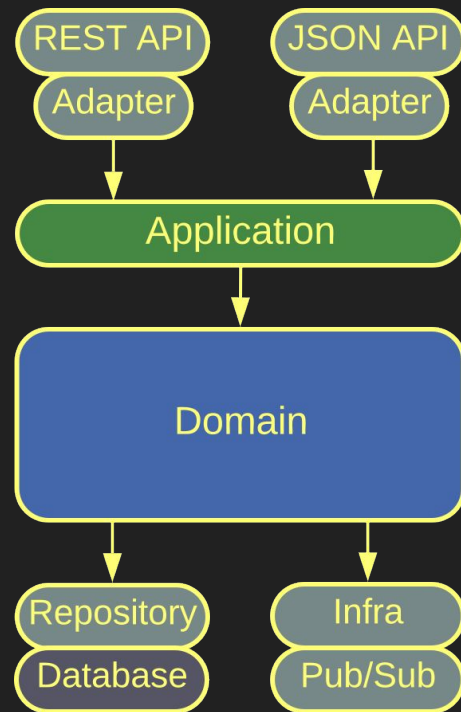
The UI can be changed without affecting the *model*.

Is easy to port to new platforms.

... or plagiarized?

Please find 3 differences between:

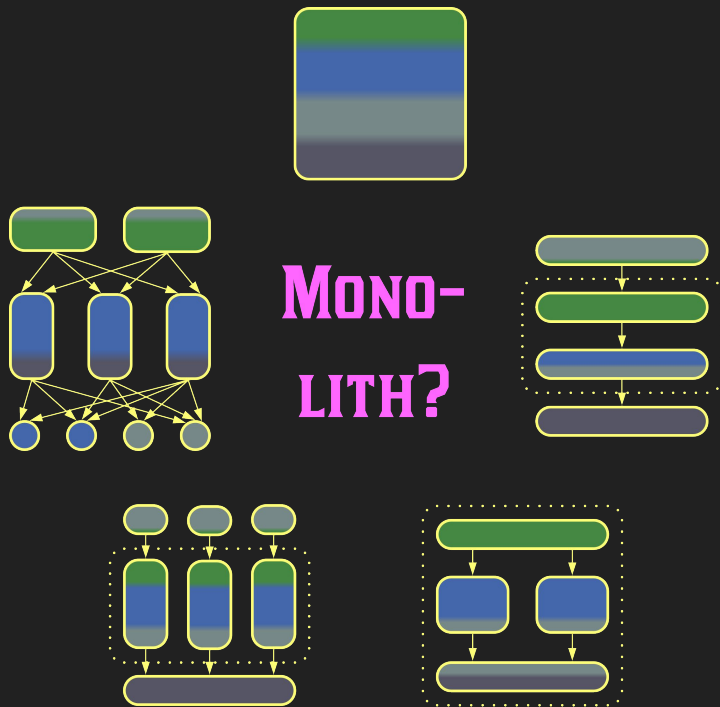
- Ports and Adapters (2005) with a DDD-style (2003) layered core.
- Onion Architecture (2008).
- Clean Architecture (2012).



Patterns change their meanings over time

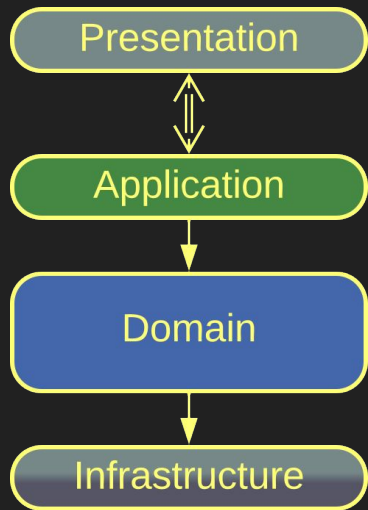
What is a *Monolith*?

- An unstructured application?
- A non-distributed system?
- A system with a shared database?
- A single unit of deployment?



Local and Distributed Architectures

Layers and Tiers

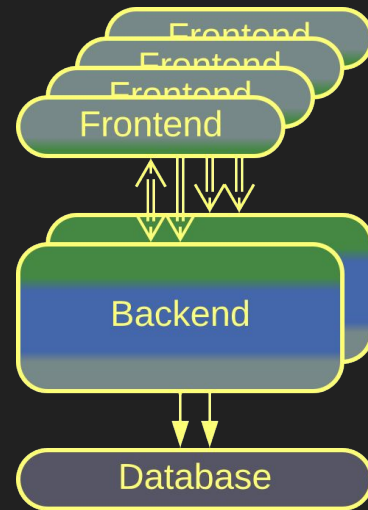


Layers

Both *Layers* and *Tiers* allow for individual components to:

- Differ in their properties (e.g. responsiveness).
- Differ in technologies (programming language).
- Be developed by dedicated teams.

Both architectures allow for a quick start of development but tend to grow unmanageable if not further partitioned by subdomain.



3-Tier

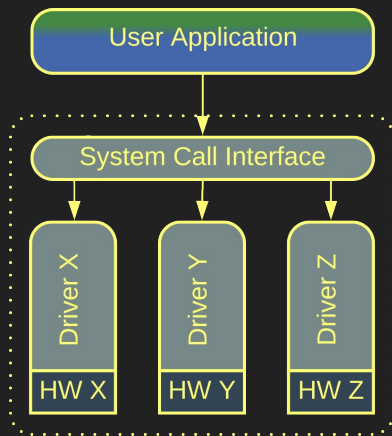
Tiers, being a distributed architecture, allow for its components to run on dedicated hardware, scale individually, differ in security, and provide a limited fault tolerance.

Device Drivers and Microservices

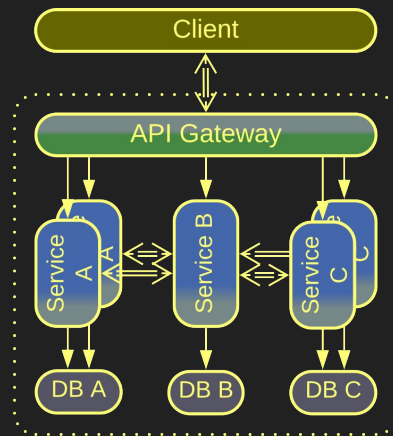
Each *device driver* and each *microservice*:

- Covers a whole subdomain.
- Is developed by a dedicated team.
- Has its own code style and technologies.
- Is tested and released independently of others.

Both architectures are dedicated to multi-team development of complex projects but necessarily sacrifice the simplicity and convenience of use cases that involve several subdomains.



OS Drivers



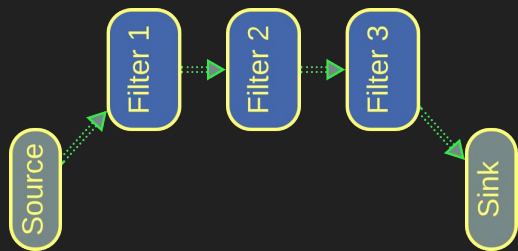
Microservices

Microservices, thanks to their distributed nature, are quite scalable and fault tolerant.

Pipes and Filters and Event-Driven Architecture

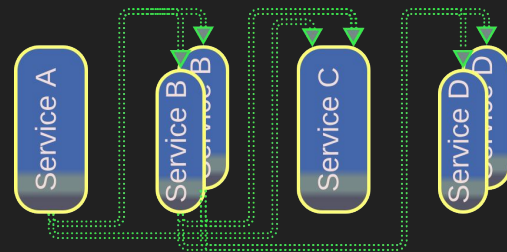
Components of *Pipes and Filters* or *Choreographed Event-Driven Architecture*:

- Are easy to scale, add or replace without affecting the whole system.
- Can be implemented and tested in isolation through event or data stream replay.
- May differ in technologies.



Pipes and Filters

Both architectures specialize in processing streams of closely related data packets or events.



Event-Driven Architecture

Choreographed Event-Driven Architecture can utilize resources of multiple servers while *Pipes and Filters* are usually limited to multiple CPU cores of a single workstation.

What have we done?

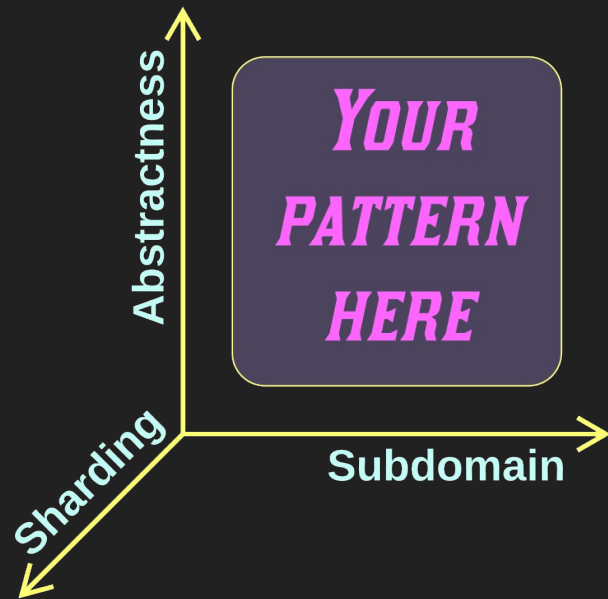
As we see, once the system properties rooted in distribution are segregated, whatever remains of a distributed architecture is identical to a decades-old pattern for local applications.

Patterns of Patterns

How did we compare architectures?

We drew them with a common system of coordinates:

- Abstractness
- Subdomain
- Sharding



Structure determines function!

Metapatterns

A metapattern is a cluster of patterns related in their structure and thus also in their properties

There are few metapatterns

As a metapattern is defined by its structure,
and the structure is a geometry in a 3D space,
there are only so many metapatterns.

Therefore hundreds of patterns are reduced
to several clusters with manifest properties.

The benefits of having a taxonomy of patterns

- A hierarchical classification is easy to learn and study.
- Duplicate or ambiguous entities become obvious.
- Novel solutions may be found through mapping architectures between domains.

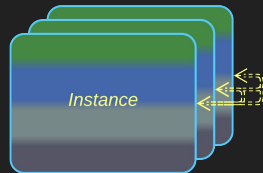
Known Metapatterns

Basic Metapatterns



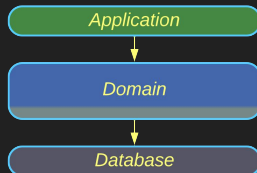
Monolith

Reactor,
Proactor,
Half-Sync /
Half-Async,
(Re)Actor-with-
Extractors



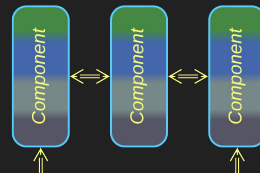
Shards

Shards,
Partitions,
Instances,
Replicas



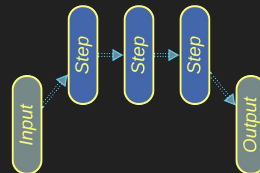
Layers

Layers, Tiers



Services

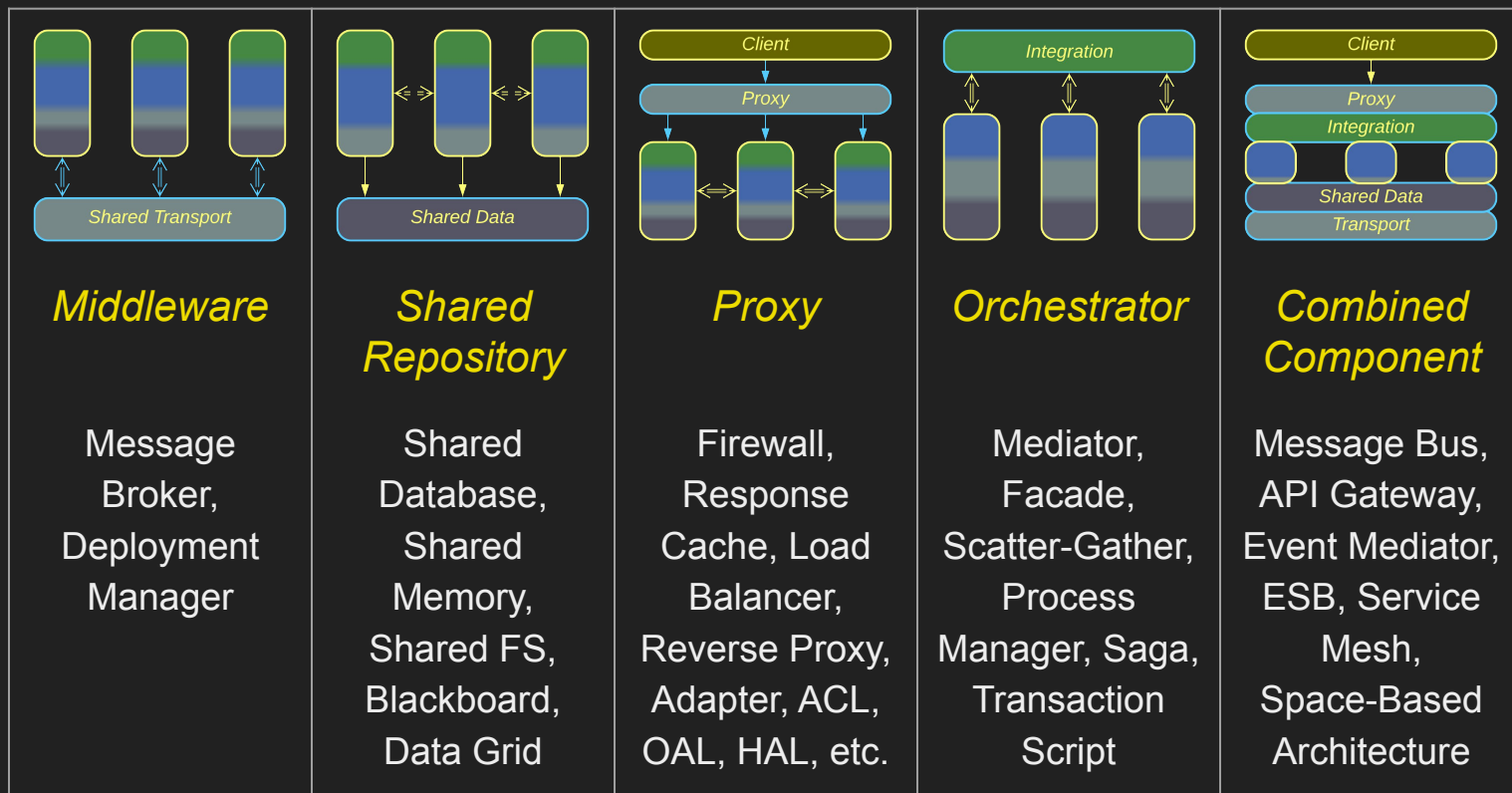
Service-Based
Architecture,
Modular
Monolith,
Microservices,
Actors



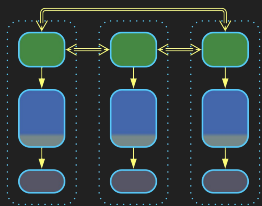
Pipeline

Pipes and
Filters,
Choreographed
Event-Driven
Architecture,
Data Mesh

Extension Metapatterns

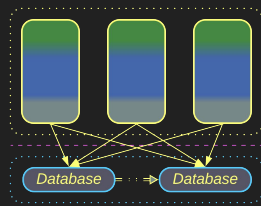


Fragmented Metapatterns



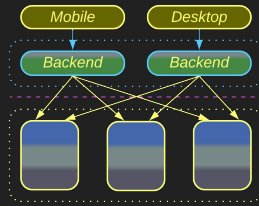
Layered Services

Orchestrated
3-Layered
Services,
Choreographed
2-Layered
Services, CQRS



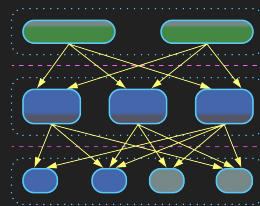
Polyglot Persistence

Specialized
Databases,
CDN, Read-Only
Replicas,
Reporting
Database



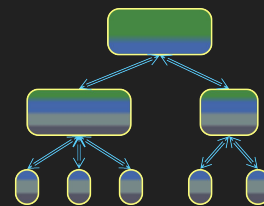
Backends for Frontends

Multiple Proxies,
Orchestrators or
API Gateways



SOA

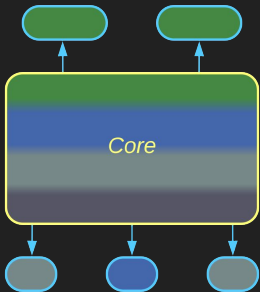
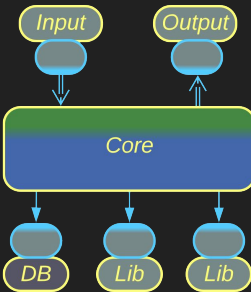
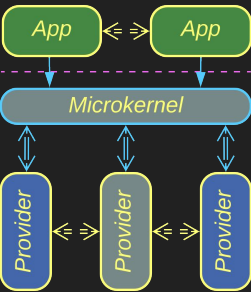
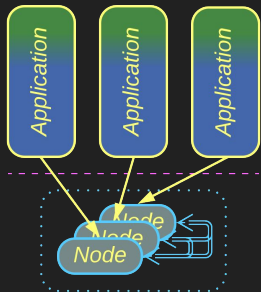
Enterprise SOA,
Domain-
Oriented
Microservice
Architecture,
Distributed
Monolith



Hierarchy

Top-Down
Hierarchy, PAC,
HMVC, Bus of
Buses, WSO2
Cell-Based
Architecture

Implementation Metapatterns

 <i>Plugins</i> Plugins, Addons, Strategy, Hooks, Aspects, Reflection	 <i>Hexagonal Architecture</i> Ports and Adapters, Onion Architecture, MVC family, MVP family	 <i>Microkernel</i> OS, Software Framework, Hypervisor, Interpreter, Saga Engine	 <i>Mesh</i> P2P Network, Actors, Service Mesh, Space-Based Architecture
--	--	--	---

Conclusion

Limitations

The structural approach works only for system-wide architectural patterns:

- No Gang of Four design patterns
- No Enterprise Integration patterns
- No TDD organizational patterns

The research is in its early phase and is not supported by any organization.

Links

My book *Architectural Metapatterns* is available:

- for reading online at metapatterns.io
- as an ebook from leanpub.com/metapatterns

The diagrams and the ODT source file are provided under the CC BY license.

**This book is a seminal research on reframing patterns.
It's up to you to see to it being recognized or discarded.**